

# Introduction to NoSQL and Key-Value Storage

## IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management  
Oklahoma State University

Fall 2026

*NoSQL material adapted from LinkedIn School of SRE,*

*<https://linkedin.github.io/school-of-sre/>. Redis material adapted from DigitalOcean,  
How To Manage a Redis Database.*

- 1 What NoSQL Is
- 2 Consistency and the CAP Theorem
- 3 Redis as a Key-Value Store

# What NoSQL Is

# What is NoSQL?

- NoSQL refers to any non-relational database, that is, any database that stores data in some format other than relational tables.
- The term appeared in the early twenty first century, although databases of this kind have existed since the late 1960s.
- A common misconception is that NoSQL databases cannot handle relationships. They can. They simply store the relationship differently than a relational system does.
- NoSQL became prominent in the late 2000s, as the cost of storage fell sharply. That shifted the focus of development away from optimizing storage and toward developer productivity, which fit the move to Agile practice at the same time.

Source: LinkedIn School of SRE

## Four Kinds of NoSQL Database, Part 1

- **Document databases** store information as JSON-like documents made of field and value pairs. A value can be a string, a number, a boolean, an array, or another object, so the stored document can mirror the structure a developer already has in code. The schema is flexible and these systems scale horizontally. MongoDB and Couchbase are examples.
- **Key-value databases** are the simplest category. A value is retrieved only through the key associated with it. They suit situations that need to store data at large scale without complex querying, such as user preferences or caching. Redis, DynamoDB, and LinkedIn's own Venice platform are examples.

Source: LinkedIn School of SRE

## Four Kinds of NoSQL Database, Part 2

- **Wide-column stores** organize data into tables with rows and dynamic columns. Rows are not required to share the same columns, which gives them flexibility a relational table does not have. They suit large datasets with predictable query patterns, and are often used for Internet of Things data and user profiles. Cassandra and HBase are examples.
- **Graph databases** store information as nodes, which are entities, and edges, which are the relationships between them. They are strong at finding patterns across connected data, which makes them useful for social networks, fraud detection, and recommendation systems. Neo4j is a leading example.

Source: LinkedIn School of SRE

## SQL and NoSQL Compared

|              | <b>SQL</b>                | <b>NoSQL</b>                                       |
|--------------|---------------------------|----------------------------------------------------|
| Structure    | Fixed rows and columns    | Documents, key-value pairs, wide columns, or nodes |
| Scaling      | Vertical, a bigger server | Horizontal, more commodity machines                |
| Schemas      | Rigid                     | Flexible                                           |
| Transactions | Native ACID support       | Limited, though some do support it                 |
| Joins        | Typically necessary       | Usually unnecessary                                |
| Data mapping | Usually needs an ORM      | Often maps directly to language structures         |

Source: LinkedIn School of SRE

# What NoSQL Buys You

- **Schema flexibility.** The structure can change without the operational overhead an `ALTER TABLE` would carry on a large relational table, which supports continuous feature development.
- **Horizontal scalability.** Data is distributed across inexpensive hardware, rather than requiring an ever more powerful single server.
- **Performance.** Denormalized data and a distributed architecture frequently make queries faster. Notice that this runs directly against the normalization we covered in Lecture 10, and it is a deliberate trade, not an oversight.
- **Developer efficiency.** Data maps directly onto programming language structures, which cuts both the transformation work and the errors that come with it.

Source: LinkedIn School of SRE

# Consistency and the CAP Theorem

# The CAP Theorem

Eric Brewer's CAP theorem identifies three properties of a distributed system.

- **Consistency.** A write made by a source is available to every reader of that shared data. Systems differ in the level of consistency they support.
- **Availability.** The system continues to respond when hardware or software failures remove some of its functionality.
- **Partition tolerance.** The system keeps working when a network partition isolates one part of it from another.

The theorem states that only two of the three can be guaranteed at the same time. Source: LinkedIn School of SRE

# The CAP Trade-off

| Choice | Character             | Examples                                    |
|--------|-----------------------|---------------------------------------------|
| C + A  | Single-site systems   | Traditional databases, LDAP                 |
| C + P  | Distributed systems   | Distributed databases,<br>locking protocols |
| A + P  | Eventually consistent | DNS, web caching                            |

- The MySQL work earlier in this course sat in the first row. A single server, strong consistency, and availability, with partitioning never in the picture.
- Once a system is distributed across machines, partitions become a fact of life rather than a choice, and the real decision is which of consistency or availability to give up when one happens.

Source: LinkedIn School of SRE

# Eventual Consistency

- Rather than insisting on strict ACID behavior, distributed systems commonly adopt eventual consistency. All readers will see a write as time goes on, but a client may briefly observe an inconsistent state while the change propagates.
- Several weaker guarantees are useful in practice.
  - **Read your own writes.** A client sees its own updates immediately, even though another client's changes may lag.
  - **Session consistency.** Updates stay visible within the scope of a session, typically while the client talks to the same server.
  - **Causal consistency.** Writes related by potential causality are seen in order by every process, though concurrent writes may appear in different orders to different processes.

Source: LinkedIn School of SRE

## Why This Material Is Here

- The NoSQL material in this lecture is drawn from LinkedIn's School of SRE, which is the public curriculum LinkedIn uses to train site reliability engineers.
- Database fundamentals of this kind, both relational and NoSQL, sit in the Level 101 core of that curriculum alongside Linux, networking, Python, systems design, and monitoring.
- If you are aiming at an SRE, DevOps, or platform engineering role, this is assumed knowledge rather than specialist knowledge. Being able to say why a system chose availability over consistency, or why a team put a cache in front of a relational database, is the kind of question these interviews ask.
- The full curriculum is open and worth your time.

<https://linkedin.github.io/school-of-sre/>

# Redis as a Key-Value Store

## Where Redis Fits

- Everything so far has been the general picture. Redis is one specific database drawn from one of those four families, the key-value family from earlier in this lecture.
- Redis holds its data in memory, which is what makes it fast and also what makes durability a separate concern rather than a given.
- It is not a replacement for the relational work we have done. In most real systems Redis sits alongside a relational database, holding the data that must be read quickly or that expires on its own.
- For the rest of this lecture, and all of the next, Redis is the concrete instance of everything described above.